SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	M HASHETHA	**Reg. No.:**	192521162
Date:	04-08-2026	**Duration:**	60 minutes

Code of Conduct

I _M.HASHETHA_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____HASHETHA M_____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.

- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.

Architecture Diagram and Component Design	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis	/5
Selection of Software Architecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and Quality Attribute Analysis	/5
Architectural Justification and Technical Presentation	/5
TOTAL	/5

SIMATS ENGINEERING

ASSESSMENT TOOL-2

CO-1

Assignment Title: Scenario-Based Requirement Analysis

AI-Powered Smart Court Case Management and Judicial Analytics Platform

COURSE: SOFTWARE ENGINEERING

COURSE CODE:CSA-1017

NAME:M.HASHETHA

REG.NO:192521162

1. System Requirement Analysis

1.1 Introduction

The AI-Powered Smart Court Case Management and Judicial Analytics Platform is designed to modernize judicial administration by integrating Artificial Intelligence (AI), Natural Language Processing (NLP), Cloud Computing, and Data Analytics into court operations. Traditional judicial systems often rely on manual documentation, paper-based records, and conventional scheduling methods, which result in delayed case processing, increased case backlogs, inefficient workload distribution, and limited transparency. To overcome these challenges, the proposed platform introduces intelligent automation and digital transformation across the entire judicial workflow.

The system enables electronic case registration, secure document management, AI-powered legal document classification, intelligent hearing scheduling, judicial workload analysis, case duration prediction, and comprehensive reporting. By automating repetitive administrative tasks and providing real-time analytical insights, the platform supports judges, lawyers, court clerks, administrators, and citizens in managing judicial processes more efficiently and transparently.

1.2 System Objectives

The primary objectives of the proposed system are:

- Digitize the complete court case management process.
- Automate legal document classification using AI and NLP.
- Predict case completion timelines using historical judicial data.
- Optimize hearing schedules by minimizing scheduling conflicts.
- Analyze judicial workload for better resource allocation.
- Generate analytical and statistical reports for court administration.
- Improve transparency by enabling citizens to track case progress.
- Reduce case backlogs and improve overall judicial efficiency.
- Ensure secure storage and management of legal records.
- Support data-driven decision-making through AI-powered analytics.

1.3 Functional Requirements Influencing the Architecture

The software architecture must support the following major functional requirements:

User Management

- User registration and secure authentication.
- Role-based access for judges, lawyers, clerks, administrators, and citizens.

Case Management

- Online case registration.
- Case modification and status updates.
- Case tracking throughout its lifecycle.

Document Management

- Upload legal documents.
- AI-based document classification.
- Secure document storage and retrieval.

Hearing Management

- Intelligent hearing scheduling.
- Judge allocation.
- Hearing calendar management.
- Automated notifications.

AI and Analytics

- Predict case completion duration.
- Analyze judicial workload.
- Generate dashboards and performance reports.

Reporting

- Generate case reports.
- Produce judicial analytics.
- Export reports in PDF or Excel format.

System Administration

- User management.
- Role management.
- Audit logging.
- Backup and recovery.

1.4 Non-Functional Requirements Influencing the Architecture

The architecture must also satisfy several quality-related requirements to ensure reliable and efficient operation.

Performance

- Fast response time.
- Efficient database access.
- Quick report generation.

Security

- User authentication.
- Role-Based Access Control (RBAC).
- Data encryption.
- Secure API communication.

Reliability

- Continuous system operation.
- Automatic backup and recovery.
- Fault tolerance.

Scalability

- Support increasing numbers of users and court cases.
- Allow future expansion to multiple courts and jurisdictions.

Availability

- 24×7 system accessibility.
- Minimal downtime during maintenance.

Maintainability

- Modular architecture.
- Easy software updates.
- Independent deployment of services.

Usability

- User-friendly interface.
- Responsive web application.
- Easy navigation for all stakeholders.

1.5 Quality Attributes Influencing the Architecture

The following quality attributes have a significant impact on selecting the software architecture:

Scalability:

The platform should efficiently handle increasing numbers of users, legal cases, and AI processing requests. The architecture must support future expansion without major modifications.

Performance:

Fast response time is essential for case registration, document retrieval, AI processing, and report generation. The architecture should minimize processing delays and optimize system resources.

Security:

Since judicial records contain highly confidential information, the architecture must implement secure authentication, authorization, encryption, audit logging, and secure communication protocols to protect sensitive data.

Reliability:

The system must remain operational even during heavy workloads or unexpected failures. Reliable backup mechanisms and fault-tolerant services are necessary to ensure uninterrupted judicial operations.

Availability:

Authorized users should be able to access the platform at any time. High availability is essential because court operations often require continuous access to case information and legal documents.

Maintainability:

The architecture should allow developers to update, test, and maintain individual components without affecting the entire system. A modular design simplifies debugging and future enhancements.

2. Selection of Software Architecture

2.1 Introduction

Selecting an appropriate software architecture is one of the most important decisions in software engineering because it determines how different components of the system are organized, communicate, and evolve over time. Since the AI-Powered Smart Court Case Management and Judicial Analytics Platform involves multiple users, AI services, cloud storage, secure document management, and integration with external judicial systems, the architecture must be scalable, secure, maintainable, and capable of supporting future enhancements.

After analyzing the system requirements and quality attributes, the most suitable architecture for the proposed system is a Hybrid Architecture, which combines Layered Architecture and Microservices Architecture.

2.2 Selected Software Architecture

Selected Architecture: Hybrid Architecture (Layered + Microservices)

The proposed system adopts a Hybrid Architecture because it combines the advantages of both Layered Architecture and Microservices Architecture.

- Layered Architecture organizes the system into logical layers such as Presentation, Business Logic, Service, and Data Access, making the application easier to understand, develop, and maintain.
- Microservices Architecture separates major functionalities such as AI document classification, hearing scheduling, notification services, and judicial analytics into independent services that can be developed, deployed, and scaled individually.

This combination provides flexibility, modularity, and better performance while supporting intelligent judicial operations.

2.3 Layers of the Proposed Architecture

The proposed architecture consists of the following layers:

1. Presentation Layer

The Presentation Layer provides the user interface through which different users interact with the system.

It includes:

- Judge Dashboard
- Lawyer Dashboard
- Court Clerk Dashboard
- Court Administrator Dashboard
- Citizen Portal
- System Administrator Dashboard

Responsibilities

- User authentication
- Display case information
- Collect user inputs

- Show reports and dashboards
- Display notifications

2. Application (Business Logic) Layer

This layer contains the core business logic of the judicial system.

It manages:

- Case registration
- Hearing scheduling
- User management
- Case tracking
- Document management
- Judicial workflow
- Report generation

It ensures that judicial rules and business processes are correctly implemented.

3. Microservices Layer

Specialized functionalities are implemented as independent microservices.

Major microservices include:

- AI Document Classification Service
- Case Prediction Service
- Judicial Analytics Service
- Notification Service
- Authentication Service

Each service operates independently and communicates through secure REST APIs.

4. Data Access Layer

The Data Access Layer acts as an interface between the application and the database.

Responsibilities include:

- Database queries
- Data validation
- Data retrieval
- Data storage

- Transaction management

This layer isolates database operations from business logic.

5. Database Layer

The Database Layer stores all judicial information securely.

It contains:

- User Database
- Case Database
- Legal Document Database
- Hearing Database
- Judgment Database
- Audit Logs
- AI Prediction Data

Cloud storage is used for storing large legal documents securely.

2.4 Why Hybrid Architecture is Suitable

The proposed judicial platform has several complex requirements that cannot be efficiently handled by a single architectural style.

The Hybrid Architecture is suitable because:

- Supports multiple user roles.
- Allows AI services to operate independently.
- Makes maintenance easier.
- Supports cloud deployment.
- Enables independent scaling of AI modules.
- Simplifies future feature additions.
- Provides better fault isolation.
- Improves system performance.
- Facilitates integration with government judicial systems.

2.5 Advantages of the Selected Architecture

The selected Hybrid Architecture offers several benefits:

Scalability

Each microservice can be scaled independently based on workload.

Maintainability

Changes in one service do not affect the entire application.

Security

Sensitive judicial data is protected through separate authentication and authorization services.

Performance

Independent services reduce processing delays and improve response time.

Reliability

If one service fails, the remaining services continue functioning.

Flexibility

New AI modules and external integrations can be added without redesigning the entire system.

2.6 Comparison with Other Architectures

Architecture	Suitability	Reason
Layered Architecture	Good	Easy to develop but limited scalability
Client-Server	Moderate	Suitable for simple applications but not AI-intensive systems
MVC	Good	Best for UI organization but insufficient for distributed AI services
Microservices	Very Good	Highly scalable but more complex to manage alone
Event-Driven	Moderate	Suitable for notifications but not complete judicial workflows
Hybrid (Layered + Microservices)	Excellent	Combines modularity, scalability, security, and maintainability

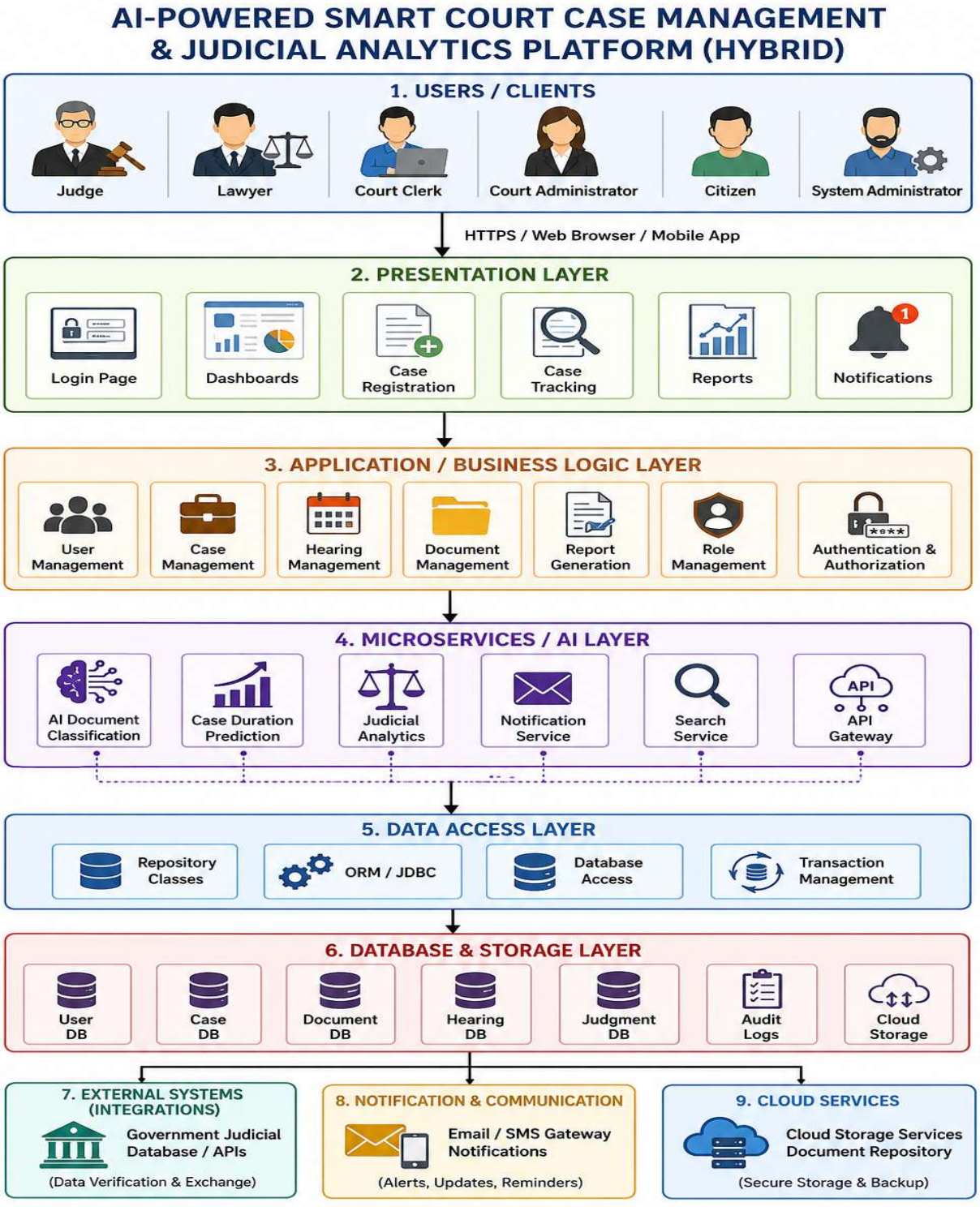
2.7 Justification for Selecting Hybrid Architecture

The AI-Powered Smart Court Case Management and Judicial Analytics Platform requires an architecture capable of managing complex judicial workflows, AI processing, secure document management, cloud

integration, and future expansion. A Hybrid Architecture effectively combines the structured organization of Layered Architecture with the flexibility and scalability of Microservices.

This architecture enables independent deployment of AI services, simplifies maintenance, improves fault tolerance, enhances security, and allows seamless integration with external government systems. It also supports future enhancements, such as virtual court hearings, AI chatbots, blockchain-based legal records, and advanced analytics, without significant architectural changes.

3.System Architecture



4. Components and Their Interactions

4.1 Introduction

The proposed AI-Powered Smart Court Case Management and Judicial Analytics Platform consists of several architectural components that work together to provide efficient, secure, and intelligent judicial services. Each component has a specific responsibility and communicates with other components through well-defined interfaces and APIs. This modular design improves system performance, scalability, maintainability, and reliability.

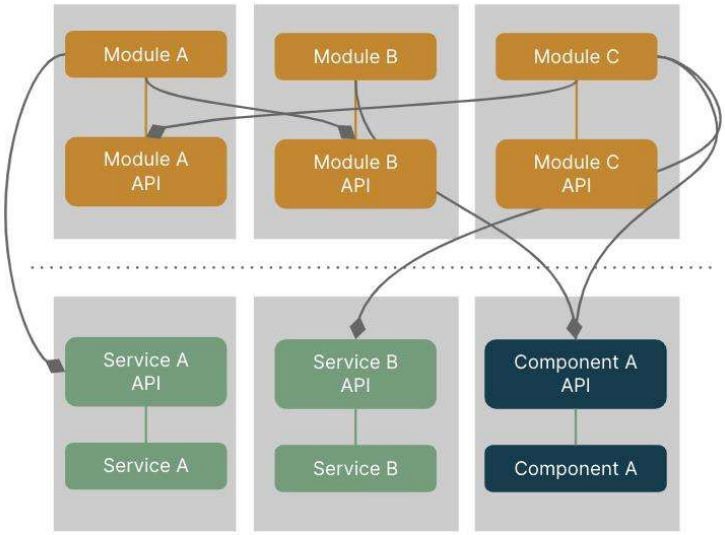
4.2 Architectural Components and Responsibilities

4.2.1 Presentation Layer

The Presentation Layer provides the graphical user interface through which all users interact with the system. It offers different dashboards and functionalities based on user roles.

Responsibilities:

- User Login and Authentication
- Role-based Dashboards
- Online Case Registration
- Case Tracking
- Hearing Schedule Display
- Report Visualization
- Notification Display
- Search and Filter Cases



Example of business modules and services interaction

Users Served:

- Judges
- Lawyers
- Court Clerks
- Court Administrators
- Citizens
- System Administrators

4.2.2 Application (Business Logic) Layer

This layer contains the core business logic that processes user requests and enforces judicial workflows.

Responsibilities:

- User Management

- Case Management
- Hearing Scheduling
- Document Management
- Workflow Processing
- Report Generation
- Role-Based Access Control (RBAC)
- Authentication and Authorization

The business layer validates user inputs and coordinates communication with AI services and databases.

4.2.3 AI and Microservices Layer

This layer contains independent services responsible for intelligent automation and specialized functionalities.

AI Document Classification Service

Automatically classifies uploaded legal documents using Artificial Intelligence and Natural Language Processing (NLP).

Case Duration Prediction Service

Analyzes historical court data to estimate the expected completion time for legal cases.

Judicial Analytics Service

Generates analytical insights, workload analysis, and statistical reports for judicial administrators.

Notification Service

Sends hearing reminders, case updates, and important announcements via email, SMS, or in-system notifications.

API Gateway

Acts as a centralized entry point for communication between internal services and external systems.

4.2.4 Data Access Layer

The Data Access Layer serves as an intermediary between the business logic and the database.

Responsibilities:

- Execute database queries
- Retrieve and store data
- Manage database transactions
- Validate data consistency
- Handle CRUD (Create, Read, Update, Delete) operations

This separation simplifies database management and improves maintainability.

4.2.5 Database and Cloud Storage Layer

This layer stores all application data securely.

Stored Data Includes:

- User Information
- Case Records
- Legal Documents
- Hearing Schedules
- Judgments
- Notifications
- Audit Logs
- AI Prediction Results

Cloud storage is used for securely storing large legal documents and maintaining backups.

4.3 Component Interactions

The components interact in the following sequence:

Step 1: User Request

A user logs into the platform and submits a request, such as registering a new case or viewing case details.

↓

Step 2: Presentation Layer

The Presentation Layer collects the user's input and forwards the request to the Business Logic Layer.

↓

Step 3: Business Logic Layer

The Business Layer validates the request, checks user permissions, and processes the required judicial workflow.

↓

Step 4: AI Services (if required)

If the request involves AI processing, such as document classification or case duration prediction, the Business Layer communicates with the appropriate Microservice through secure APIs.

↓

Step 5: Data Access Layer

The processed information is sent to the Data Access Layer, which performs the necessary database operations.

↓

Step 6: Database Layer

The required data is stored or retrieved from the database and cloud storage.

↓

Step 7: Response

The processed result is returned through the Business Layer to the Presentation Layer, where it is displayed to the user.

4.4 Overall Workflow

The complete workflow of the Smart Court Platform is as follows:

1. User logs into the system.
2. User selects the required service.
3. Business logic validates the request.
4. AI services perform intelligent processing when required.
5. Data is stored or retrieved from the database.
6. Notifications are sent to relevant stakeholders.
7. Reports and dashboards are updated.
8. The user receives the requested information through the interface.

4.5 Communication Between Components

Source Component	Destination Component	Communication Method
User	Presentation Layer	HTTPS / Web Browser
Presentation Layer	Business Logic Layer	HTTP/REST API
Business Logic Layer	AI Microservices	REST APIs
Business Logic Layer	Data Access Layer	Internal Method Calls
Data Access Layer	Database	SQL Queries / ORM
Business Logic Layer	Email/SMS Gateway	REST API
Business Logic Layer	Government Judicial Systems	Secure REST APIs

5. Quality Attribute Analysis

5.1 Introduction

Quality attributes define the characteristics that determine the effectiveness and reliability of a software system beyond its functional capabilities. For the **AI-Powered Smart Court Case Management and Judicial Analytics Platform**, the selected **Hybrid Architecture (Layered + Microservices)** is designed to satisfy important quality attributes such as scalability, security, performance, reliability, maintainability, and availability. These attributes ensure that the platform can efficiently support judicial operations while remaining secure, robust, and adaptable to future requirements.

5.2 Scalability

Scalability refers to the system's ability to handle increasing numbers of users, court cases, and transactions without affecting performance.

The proposed Hybrid Architecture supports scalability by separating major functionalities into independent microservices. Each service, such as AI document classification, hearing scheduling, and judicial analytics, can be scaled individually according to demand. Cloud infrastructure further enables the system to accommodate growing workloads and future expansion to multiple courts or judicial regions.

How the architecture supports scalability:

- Independent scaling of microservices.
- Cloud-based deployment.
- Efficient resource utilization.
- Supports increasing users and case records.

5.3 Security

Security is essential because the system stores confidential judicial records and sensitive personal information.

The architecture incorporates multiple security mechanisms, including user authentication, role-based access control (RBAC), data encryption, secure API communication, and audit logging. These measures ensure that only authorized users can access sensitive information while protecting data from unauthorized access and cyber threats.

Security features include:

- Secure user authentication.
- Role-Based Access Control (RBAC).
- SSL/TLS encrypted communication.

- Encrypted database storage.
- Audit logs for monitoring system activities.

5.4 Performance

Performance measures how efficiently the system processes user requests and responds under varying workloads.

The architecture improves performance by distributing specialized tasks among independent services. AI processing, database operations, and notification services run separately, reducing the workload on the main application. Optimized database access and cloud resources help minimize response time and improve overall system efficiency.

Performance improvements:

- Faster response time.
- Efficient database queries.
- Independent AI processing.
- Optimized report generation.

5.5 Reliability

Reliability refers to the system's ability to operate correctly and consistently without unexpected failures. The Hybrid Architecture improves reliability through independent services and fault isolation. If one microservice experiences an issue, other services continue functioning without interrupting the entire platform. Regular backups and cloud-based storage further enhance data protection and system recovery.

Reliability mechanisms:

- Fault isolation between services.
- Automatic data backup.
- Error handling and recovery.
- Continuous monitoring of services.

5.6 Maintainability

Maintainability is the ease with which the software can be updated, tested, and modified.

The layered design separates presentation, business logic, and data management, while microservices isolate specialized functionalities. This modular approach allows developers to update or replace individual components without affecting the rest of the system, reducing maintenance time and improving software quality.

Maintainability benefits:

- Modular software design.
- Independent service updates.
- Easier debugging and testing.
- Simplified future enhancements.

5.7 Availability

Availability ensures that the system remains accessible whenever authorized users require its services. Cloud-based deployment, database backup mechanisms, and fault-tolerant services help maintain continuous system availability. Even during maintenance or increased workload, essential judicial services remain operational with minimal downtime.

Availability features:

- 24×7 system accessibility.
- Cloud infrastructure support.
- Backup and disaster recovery.
- Minimal service interruption.

6. Architectural Justification

6.1 Introduction

The selection of an appropriate software architecture is a critical step in developing a reliable and scalable system. The **AI-Powered Smart Court Case Management and Judicial Analytics Platform** requires an architecture capable of handling complex judicial workflows, AI-based services, cloud integration, secure data management, and future technological advancements. After analyzing the system requirements and quality attributes, the **Hybrid Architecture (Layered + Microservices)** was selected as the most suitable architectural style.

6.2 Rationale for Selecting Hybrid Architecture

The Hybrid Architecture combines the strengths of **Layered Architecture** and **Microservices Architecture**.

- The **Layered Architecture** organizes the application into Presentation, Business Logic, Data Access, and Database layers, making the system easy to understand, develop, and maintain.

- The **Microservices Architecture** separates specialized functionalities, such as AI document classification, case duration prediction, judicial analytics, and notification services, into independent services.

This combination provides flexibility, modularity, and efficient management of complex judicial operations while ensuring the system can grow and adapt to future requirements.

6.3 Trade-Offs Considered

Although the Hybrid Architecture provides many advantages, certain trade-offs were considered during its selection.

Advantages

- High scalability through independent microservices.
- Better maintainability due to modular design.
- Improved fault isolation, ensuring one service failure does not affect the entire system.
- Easier integration with AI modules and external government systems.
- Enhanced flexibility for adding future features.
- Strong support for cloud deployment and distributed systems.

Disadvantages

- More complex system design compared to a simple layered architecture.
- Higher deployment and monitoring effort due to multiple services.
- Increased communication overhead between microservices.
- Requires efficient API management and service coordination.

Despite these challenges, the advantages significantly outweigh the limitations for a large-scale judicial platform.

6.4 Support for Future Enhancements

The selected architecture is designed to accommodate future technological advancements without requiring major modifications.

Possible future enhancements include:

- Virtual Court and Video Hearing Integration.
- AI-powered Legal Chatbot for citizen assistance.
- Blockchain-based legal document verification.
- Mobile applications for Android and iOS.
- Voice-to-Text transcription of court proceedings.
- Multilingual user interface.
- Integration with additional government legal databases.

- Advanced AI models for legal recommendation and case prioritization.

Because each service operates independently, new functionalities can be added with minimal impact on the existing system.

6.5 Support for System Integration

The Hybrid Architecture enables seamless integration with external systems through secure REST APIs. The platform can integrate with:

- Government Judicial Information Systems.
- National Legal Databases.
- Email Notification Services.
- SMS Gateway Services.
- Cloud Storage Providers.
- AI and Machine Learning Platforms.
- Digital Signature Services.
- E-Governance Portals.

These integrations improve interoperability and allow secure exchange of judicial information.

6.6 Long-Term Maintainability

Long-term software maintenance is simplified through modular system design.

The architecture supports:

- Independent testing of each microservice.
- Easy software updates.
- Faster bug fixing.
- Component replacement without affecting the complete system.
- Better code reusability.
- Continuous integration and deployment.

This reduces maintenance costs and improves software quality throughout the system lifecycle.

6.8 Conclusion

The **Hybrid Architecture (Layered + Microservices)** is the most suitable architectural style for the **AI-Powered Smart Court Case Management and Judicial Analytics Platform** because it successfully addresses both functional and non-functional requirements. It provides a secure, scalable, reliable, and maintainable framework capable of supporting intelligent judicial services and future technological advancements.

By combining the structured organization of layered architecture with the flexibility of microservices, the proposed solution enables efficient case management, AI-powered analytics, secure document handling, and seamless integration with external systems.